

AI-Driven Emergency Care Coordination with Syndromic Surveillance

- ***Author:*** Katharina, MD
- ***Date:*** 2025-08-29
- ***Setting:*** Urban ED (research sandbox, deidentified data)
- --

1. Introduction

Emergency departments (EDs) must coordinate rapid diagnostics and multi-team handoffs under variable load. Common pain points include: (i) information loss across handovers (EMS → triage → ED physician → specialty teams), (ii) manual orchestration of CT/X-ray/labs/point-of-care ultrasound, (iii) bottlenecks in bed allocation and ICU capacity, and (iv) high cognitive and administrative burden. We built a research pipeline that integrates an existing machine learning (MLP) gate engine with a ****syndromic surveillance**** layer trained on deidentified MIMIC-IV-ED data to surface emerging patterns in near-real time while preserving current clinical workflows.

- ***Syndromic*** here means tracking symptom clusters (e.g., respiratory or GI) from chief complaints, triage, and vitals before a definitive diagnosis is available. The objective is early situational awareness, not diagnosis.
- --

2. Related Work (brief)

- Queueing and operational analytics in EDs (arrival/boarding/ICU bottlenecks).
- Early warning systems for respiratory/GI surges and influenza-like illness.
- ED MLP/risk-stratification pipelines that support gate-based recommendations.

Our contribution is a ****minimal-intrusion modular layer**** that plugs into an existing MLP/gate UI and uses only deidentified data (MIMIC-IV-ED demo) with transparent LLM-assisted development.

- --

3. Data and Privacy

- ***Dataset:*** MIMIC-IV-ED demo tables: `edstays`, `triage`, `diagnosis`, `vitalsign` (optional `medrecon`, `pyxis`). All inputs are deidentified prior to release.
- ***Identifiers:*** The pipeline operates on `stay\_id` and hourly time buckets. No real-world identifiers are used or exported.
- ***PHI/GDPR safeguards:*** Free-text is tokenized to counts; dates are bucketed to hour; ages can be binned (e.g., 90+); logs use pseudonyms and omit names/IP/email. No reidentification attempts are

permitted.

- ***Paths (Kaggle):***
- Data root: ``/kaggle/input/mimic-iv-demo-v2-2``
- Surveillance module: ``/kaggle/input/ed-surveillance/ed_surveillance_system.py``
- Gate/MLP support: ``/kaggle/input/edtracker-ops-pg``, ``/kaggle/input/edtracker-ui-pkg``, ``/kaggle/input/tracker-core``
- --

4. System Architecture

- 1 ****UI + Gate Engine (existing):*** clinical rules and MLP bundle drive care pathways and recommendations.
- 2 ****Syndromic Surveillance (new):*** feature builder + anomaly detector running on the same deidentified data; produces alerts (respiratory/ GI / acuity shifts / volume surges) and a realtime score.
- 3 ****Equipment/ops telemetry:*** (optional) device/location tracking and SOP registry; logically separate from patient data.

The layer communicates via shared data structures; no changes to existing gate code are required.

- --

5. Feature Engineering

Hourly features (index: ``ts``):

- ****Arrivals:*** unique ``stay_id`` per hour.
- ****Occupancy:*** interval expansion from ``intime`` → ``outtime``.
- ****Demographics:*** gender counts; age median and 90th percentile (if available).
- ****Triage/Acuity:*** ESI distribution per hour.
- ****Chiefcomplaint tokens:*** topK vocabulary from ``triage.chiefcomplaint``.
- ****Vitals:*** hourly medians/means (HR, RR, SpO₂, Temp, SBP/DBP) when present.
- ****Seasonality:*** sin/cos encodings for dayofweek and hour.

Implementation uses robust readers for ``csv`/`.csv.gz``, defensive datetime coercion, and consistent column naming. Realtime windows reuse exactly the same builder to ensure column alignment.

- --

6. Models

6.1 Existing Gate MLP

The MLP classifier (loaded from a serialized bundle) underpins gate recommendations. It is left untouched.

6.2 Syndromic Detector (Unsupervised)

A lightweight scikit-learn pipeline:

...

SimpleImputer(strategy="median") → StandardScaler → IsolationForest

...

- **Training:** fit on historical hourly feature matrix.
- **Scoring:** `score\_samples` converted to anomaly score; higher = more anomalous.
- **Alerts:** keyword-based buckets (respiratory, GI) plus seasonal heuristics produce “info/warning/critical” messages with recommended actions.

This design is dataset-agnostic, fast, and easy to calibrate.

- --

7. Implementation Notes

- Reproducible paths and imports; smoke tests for bundle load, gating, feature build, and real-time score.
- Imputer + scaler handle NaNs and scale drift; the same feature order is enforced at inference.
- Event logging writes pseudonymous, hourly-bucketed records to a local JSONL file.
- UI integration exposes surveillance alerts without altering gate logic.

- --

8. Results (demo runs)

On the MIMIC-IV demo set, end-to-end execution produced stable features (~ 50+ columns) and anomaly scoring across the full timeline. The detector yielded plausible alerts (e.g., GI warnings) and a consistent one-hour real-time score at the latest bucket. Since this is deidentified research data, these are **method feasibility** results rather than prospective clinical outcomes.

- --

9. Evaluation Plan (prospective)

- **Retrospective calibration:** align alert thresholds to local baselines; backtest against historical peaks (flu seasons, GI outbreaks).
- **Prospective trial:** silent mode → clinician-in-the-loop → operational piloting.

- **Metrics:** alert precision/recall (by syndrome), lead time vs. diagnosis confirmations, effect on boarding time and resource allocation, alert fatigue.
- **Fairness & safety:** subgroup performance (age, sex), monitoring for spurious correlations, governance review.
- --

10. Limitations

- Deidentified public data ≠ local case mix; thresholds require site-specific tuning.
- Keyword-based syndrome mapping can miss multilingual/colloquial expressions; NLP upgrades (e.g., curated lexicons) are planned.
- Unsupervised detectors flag anomalies, not causes; interpret with clinical context.
- --

11. Transparency & Ethics

- **Authorship:** physician domain expert led design and validation.
- **LLM use:** code refactoring, error handling, documentation, and drafting support. All outputs were executed on deidentified data and reviewed.
- **Data protection:** no PHI; logs use pseudonyms and hourly buckets; no external data exports.
- **Open science:** modular code, clear interfaces, and documented configuration.
- --

12. Conclusion

A drop-in syndromic layer can strengthen ED coordination by surfacing early pattern shifts without disrupting existing MLP-driven gates or UI. The architecture is lightweight, privacy-preserving, and ready for site-specific calibration and prospective evaluation.

• --

Appendix A. Reproducibility (key paths)

- Data: ``/kaggle/input/mimic-iv-demo-v2-2``
- Surveillance module: ``/kaggle/input/ed-surveillance/ed_surveillance_system.py``
- UI/ops packages: ``/kaggle/input/edtracker-ops-pg``, ``/kaggle/input/edtracker-ui-pkg``, ``/kaggle/input/tracker-core``

Appendix B. Minimal training/RT code (pseudocode)

```
```python
from ed_surveillance_system import MIMICSurveillanceData, IsolationForestSurveillance

ds = MIMICSurveillanceData(data_path="/kaggle/input/mimic-iv-demo-v2-2")
feats = ds.get_surveillance_features(freq="H", top_k_complaints=25)

model = IsolationForestSurveillance(random_state=42, contamination="auto").fit(feats)
detections = model.detect(feats, top_n=20)
rt = ds.make_realtime_features(window="1H")
rt_score = model.score(rt)
```
```